

Problem A. NIT orz!

Time Limit 1000 ms

Mem Limit 524288 kB

NIT, the cleaver, is new in town! Thousands of people line up to orz him. To keep his orzers entertained, NIT decided to let them solve the following problem related to or z . Can you solve this problem too?

You are given a 1-indexed array of n integers, a , and an integer z . You can do the following operation any number (possibly zero) of times:

- Select a positive integer i such that $1 \leq i \leq n$. Then, **simultaneously** set a_i to $(a_i \text{ or } z)$ and set z to $(a_i \text{ and } z)$. In other words, let x and y respectively be the current values of a_i and z . Then set a_i to $(x \text{ or } y)$ and set z to $(x \text{ and } y)$.

Here or and and denote the [bitwise operations OR and AND](#) respectively.

Find the maximum possible value of the maximum value in a after any number (possibly zero) of operations.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 100$). Description of the test cases follows.

The first line of each test case contains two integers n and z ($1 \leq n \leq 2000, 0 \leq z < 2^{30}$).

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i < 2^{30}$).

It is guaranteed that the sum of n over all test cases does not exceed 10^4 .

Output

For each test case, print one integer — the answer to the problem.

Examples

Input	Output
5 2 3 3 4 5 5 0 2 4 6 8 1 9 10 5 7 7 15 30 29 27 3 39548743 10293834 10284344 13635445	7 13 11 31 48234367

Note

In the first test case of the sample, one optimal sequence of operations is:

- Do the operation with $i = 1$. Now a_1 becomes $(3 \text{ or } 3) = 3$ and z becomes $(3 \text{ and } 3) = 3$.
- Do the operation with $i = 2$. Now a_2 becomes $(4 \text{ or } 3) = 7$ and z becomes $(4 \text{ and } 3) = 0$.
- Do the operation with $i = 1$. Now a_1 becomes $(3 \text{ or } 0) = 3$ and z becomes $(3 \text{ and } 0) = 0$.

After these operations, the sequence a becomes $[3, 7]$, and the maximum value in it is 7. We can prove that the maximum value in a can never exceed 7, so the answer is 7.

In the fourth test case of the sample, one optimal sequence of operations is:

- Do the operation with $i = 1$. Now a_1 becomes $(7 \text{ or } 7) = 7$ and z becomes $(7 \text{ and } 7) = 7$.
- Do the operation with $i = 3$. Now a_3 becomes $(30 \text{ or } 7) = 31$ and z becomes $(30 \text{ and } 7) = 6$.
- Do the operation with $i = 5$. Now a_5 becomes $(27 \text{ or } 6) = 31$ and z becomes $(27 \text{ and } 6) = 2$.

Problem B. XORwice

Time Limit 1000 ms

Mem Limit 262144 kB

In order to celebrate Twice's 5th anniversary, Tzuyu and Sana decided to play a game.

Tzuyu gave Sana two integers a and b and a really important quest.

In order to complete the quest, Sana has to output the smallest possible value of $(a \oplus x) + (b \oplus x)$ for any given x , where $\oplus$ denotes the [bitwise XOR operation](#).

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 10^4$). Description of the test cases follows.

The only line of each test case contains two integers a and b ($1 \leq a, b \leq 10^9$).

Output

For each testcase, output the smallest possible value of the given expression.

Examples

Input	Output
6	10
6 12	13
4 9	891
59 832	18
28 14	6237
4925 2912	0
1 1	

Note

For the first test case Sana can choose $x = 4$ and the value will be $(6 \oplus 4) + (12 \oplus 4) = 2 + 8 = 10$. It can be shown that this is the smallest possible value.

Problem C. LLPS

Time Limit 2000 ms

Mem Limit 262144 kB

Input File stdin

Output File stdout

This problem's actual name, "Lexicographically Largest Palindromic Subsequence" is too long to fit into the page headline.

You are given string S consisting of lowercase English letters only. Find its lexicographically largest palindromic subsequence.

We'll call a non-empty string $s[p_1 p_2 \dots p_k] = s_{p_1} s_{p_2} \dots s_{p_k}$ ($1 \leq p_1 < p_2 < \dots < p_k \leq |S|$) a subsequence of string $S = s_1 s_2 \dots s_{|S|}$, where $|S|$ is the length of string S . For example, strings "abcb", "b" and "abacaba" are subsequences of string "abacaba".

String $x = x_1 x_2 \dots x_{|x|}$ is lexicographically larger than string $y = y_1 y_2 \dots y_{|y|}$ if either $|x| > |y|$ and $x_1 = y_1, x_2 = y_2, \dots, x_{|y|} = y_{|y|}$, or there exists such number r ($r < |x|, r < |y|$) that $x_1 = y_1, x_2 = y_2, \dots, x_r = y_r$ and $x_{r+1} > y_{r+1}$. Characters in the strings are compared according to their ASCII codes. For example, string "ranger" is lexicographically larger than string "racecar" and string "poster" is lexicographically larger than string "post".

String $S = s_1 s_2 \dots s_{|S|}$ is a palindrome if it matches string $rev(S) = s_{|S|} s_{|S|-1} \dots s_1$. In other words, a string is a palindrome if it reads the same way from left to right and from right to left. For example, palindromic strings are "racecar", "refer" and "z".

Input

The only input line contains a non-empty string S consisting of lowercase English letters only. Its length does not exceed 10.

Output

Print the lexicographically largest palindromic subsequence of string S .

Examples

Input	Output
radar	rr

Input	Output
bowwowwow	wwwww

Input	Output
codeforces	s

Input	Output
mississipp	ssss

Note

Among all distinct subsequences of string "radar" the following ones are palindromes: "a", "d", "r", "aa", "rr", "ada", "rar", "rdr", "raar" and "radar". The lexicographically largest of them is "rr".

Problem D. Patchouli's Magical Talisman

Time Limit 1000 ms

Mem Limit 262144 kB

She is skilled in all kinds of magics, and is keen on inventing new one.

—*Perfect Memento in Strict Sense*

Patchouli is making a magical talisman. She initially has n magical tokens. Their magical power can be represented with **positive** integers $a_1, a_2, \dots, a_n$.

Patchouli may perform the following two operations on the tokens.

- **Fusion:** Patchouli chooses two tokens, removes them, and creates a new token with magical power equal to the sum of the two chosen tokens.
- **Reduction:** Patchouli chooses a token with an **even** value of magical power x , removes it and creates a new token with magical power equal to $\frac{x}{2}$.

Tokens are more effective when their magical powers are **odd** values. Please help Patchouli to find the minimum number of operations she needs to make magical powers of all tokens **odd** values.

Input

Each test contains multiple test cases.

The first line contains a single integer t ($1 \leq t \leq 10^3$) — the number of test cases. The description of the test cases follows.

For each test case, the first line contains one integer n ($1 \leq n \leq 2 \cdot 10^5$) — the initial number of tokens.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — the initial magical power of the n tokens.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print a single integer — the minimum number of operations Patchouli needs to make all tokens have an **odd** value of magical power.

It can be shown that under such restrictions the required sequence of operations exists.

Examples

Input	Output
4 2 1 9 3 1 1 2 3 2 4 8 3 1049600 33792 1280	0 1 3 10

Note

Test case 1:

a consists solely of odd numbers initially.

Test case 2:

Choose the tokens with magical power of 1 and 2 and perform Fusion. Now $a = [1, 3]$, both are odd numbers.

Test case 3:

Choose the tokens with magical power of 2 and 8 and perform Fusion. Now $a = [4, 10]$.

Choose the token with magical power of 10 and perform Reduction. Now $a = [4, 5]$.

Choose the tokens with magical power of 4 and 5 and perform Fusion. Now $a = [9]$, and 9 is an odd number.

It can be shown that you can not make all the magical powers **odd** numbers in less than 3 moves, so the answer is 3.

Problem E. Cirno's Perfect Bitmasks Classroom

Time Limit 1000 ms

Mem Limit 262144 kB

Even if it's a really easy question, she won't be able to answer it

— *Perfect Memento in Strict Sense*

Cirno's perfect bitmasks classroom has just started!

Cirno gave her students a positive integer x . As an assignment, her students need to find the **minimum positive integer** y , which satisfies the following two conditions:

$$x \text{ and } y > 0$$

$$x \text{ xor } y > 0$$

Where **and** is the [bitwise AND operation](#), and **xor** is the [bitwise XOR operation](#).

Among the students was Mystia, who was truly baffled by all these new operators. Please help her!

Input

The first line of input contains a single integer t ($1 \leq t \leq 10^3$) — the number of input test cases.

For each test case, the only line of input contains one integer x ($1 \leq x \leq 2^{30}$).

Output

For each test case, print a single integer — the minimum number of y .

Examples

Input	Output
7 1 2 5 9 16 114514 1000000	3 3 1 1 17 2 64

Note

Test case 1:

$1 \text{ and } 3 = 1 > 0$, $1 \text{ xor } 3 = 2 > 0$.

Test case 2:

$2 \text{ and } 3 = 2 > 0$, $2 \text{ xor } 3 = 1 > 0$.

Problem F. Ancient Civilization

Time Limit 1000 ms

Mem Limit 262144 kB

Martian scientists explore Ganymede, one of Jupiter's numerous moons. Recently, they have found ruins of an ancient civilization. The scientists brought to Mars some tablets with writings in a language unknown to science.

They found out that the inhabitants of Ganymede used an alphabet consisting of two letters, and each word was exactly ℓ letters long. So, the scientists decided to write each word of this language as an integer from 0 to $2^\ell - 1$ inclusively. The first letter of the alphabet corresponds to zero bit in this integer, and the second letter corresponds to one bit.

The same word may have various forms in this language. Then, you need to restore the initial form. The process of doing it is described below.

Denote the *distance* between two words as the amount of positions, in which these words differ. For example, the distance between 1001_2 and 1100_2 (in binary) is equal to two, as these words have different letters in the second and the fourth positions, counting from left to right. Further, denote the distance between words x and y as $d(x, y)$.

Let the word have n forms, the i -th of which is described with an integer x_i . All the x_i are not necessarily different, as two various forms of the word can be written the same.

Consider some word y . Then, *closeness* of the word y is equal to the sum of distances to each of the word forms, i. e. the sum $d(x_i, y)$ over all $1 \leq i \leq n$.

The initial form is the word y with minimal possible nearness.

You need to help the scientists and write the program which finds the initial form of the word given all its known forms. Note that the initial form is **not necessarily** equal to any of the n given forms.

Input

The first line contains an integer t ($1 \leq t \leq 100$) — the number of test cases. The following are descriptions of the test cases.

The first line contains two integers n and ℓ ($1 \leq n \leq 100$, $1 \leq \ell \leq 30$) — the amount of word forms, and the number of letters in one word.

The second line contains n integers x_i ($0 \leq x_i \leq 2^\ell - 1$) — word forms. The integers are not necessarily different.

Output

For each test, print a single integer, the initial form of the word, i. e. such y ($0 \leq y \leq 2^\ell - 1$) that the sum $d(x_i, y)$ over all $1 \leq i \leq n$ is minimal possible. Note that y can differ from all the integers x_i .

If there are multiple ways to restore the initial form, print any.

Examples

Input	Output
7 3 5 18 9 21 3 5 18 18 18 1 1 1 5 30 1 2 3 4 5 6 10 99 35 85 46 78 55 2 1 0 1 8 8 5 16 42 15 83 65 78 42	17 18 1 1 39 0 2

Note

In the first test case, the words can be written as $x_1 = 10010_2$, $x_2 = 01001_2$ and $x_3 = 10101_2$ in binary. Let $y = 10001_2$. Then, $d(x_1, y) = 2$ (the difference is in the fourth and the fifth positions), $d(x_2, y) = 2$ (the difference is in the first and the second positions), $d(x_3, y) = 1$ (the difference is in the third position). So, the closeness is $2 + 2 + 1 = 5$. It can be shown that you cannot achieve smaller closeness.

In the second test case, all the forms are equal to 18 (10010_2 in binary), so the initial form is also 18. It's easy to see that closeness is equal to zero in this case.

Problem G. MIN-MEX Cut

Time Limit 1000 ms

Mem Limit 262144 kB

A binary string is a string that consists of characters 0 and 1.

Let MEX of a binary string be the smallest digit among 0, 1, or 2 that does not occur in the string. For example, MEX of 001011 is 2, because 0 and 1 occur in the string at least once, MEX of 1111 is 0, because 0 and 2 do not occur in the string and $0 < 2$.

A binary string s is given. You should cut it into any number of substrings such that each character is in exactly one substring. It is possible to cut the string into a single substring — the whole string.

A string a is a substring of a string b if a can be obtained from b by deletion of several (possibly, zero or all) characters from the beginning and several (possibly, zero or all) characters from the end.

What is the **minimal** sum of MEX of all substrings pieces can be?

Input

The input consists of multiple test cases. The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases. Description of the test cases follows.

Each test case contains a single binary string s ($1 \leq |s| \leq 10^5$).

It's guaranteed that the sum of lengths of s over all test cases does not exceed 10^5 .

Output

For each test case print a single integer — the minimal sum of MEX of all substrings that it is possible to get by cutting s optimally.

Examples

Input	Output
6 01 1111 01100 101 0000 01010	1 0 2 1 1 2

Note

In the first test case the minimal sum is $\text{MEX}(0) + \text{MEX}(1) = 1 + 0 = 1$.

In the second test case the minimal sum is $\text{MEX}(1111) = 0$.

In the third test case the minimal sum is $\text{MEX}(01100) = 2$.

Problem H. Not Quite Latin Square

Time Limit 1000 ms

Mem Limit 262144 kB

A *Latin square* is a 3×3 grid made up of the letters A, B, and C such that:

- in each row, the letters A, B, and C each appear once, and
- in each column, the letters A, B, and C each appear once.

For example, one possible Latin square is shown below.

$$\begin{bmatrix} A & B & C \\ C & A & B \\ B & C & A \end{bmatrix}$$

You are given a Latin square, but one of the letters was replaced with a question mark ?. Find the letter that was replaced.

Input

The first line of the input contains a single integer t ($1 \leq t \leq 108$) — the number of testcases.

Each test case contains three lines, each consisting of three characters, representing the Latin square. Each character is one of A, B, C, or ?.

Each test case is a Latin square with exactly one of the letters replaced with a question mark ?.

Output

For each test case, output the letter that was replaced.

Examples

Input	Output
3 ABC C?B BCA BCA CA? ABC ?AB BCA ABC	A B C

Note

The correct Latin squares for the three test cases are shown below:

$$\begin{bmatrix} A & B & C \\ C & \textcolor{red}{A} & B \\ B & C & A \end{bmatrix} \quad \begin{bmatrix} B & C & A \\ C & A & \textcolor{red}{B} \\ A & B & C \end{bmatrix} \quad \begin{bmatrix} \textcolor{red}{C} & A & B \\ B & C & A \\ A & B & C \end{bmatrix}$$

Problem I. We Need the Zero

Time Limit 1000 ms

Mem Limit 262144 kB

There is an array a consisting of non-negative integers. You can choose an integer x and denote $b_i = a_i \oplus x$ for all $1 \leq i \leq n$, where $\oplus$ denotes the [bitwise XOR operation](#). Is it possible to choose such a number x that the value of the expression $b_1 \oplus b_2 \oplus \dots \oplus b_n$ equals 0?

It can be shown that if a valid number x exists, then there also exists x such that $(0 \leq x < 2^8)$.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 1000$). The description of the test cases follows.

The first line of the test case contains one integer n ($1 \leq n \leq 10^3$) — the length of the array a .

The second line of the test case contains n integers — array a ($0 \leq a_i < 2^8$).

It is guaranteed that the sum of n over all test cases does not exceed 10^3 .

Output

For each set test case, print the integer x ($0 \leq x < 2^8$) if it exists, or -1 otherwise.

Examples

Input	Output
5 3 1 2 5 3 1 2 3 4 0 1 2 3 4 1 2 2 3 1 1	6 0 3 -1 1

Note

In the first test case, after applying the operation with the number 6 the array b becomes $[7, 4, 3]$, $7 \oplus 4 \oplus 3 = 0$.

There are other answers in the third test case, such as the number 0.

Problem J. Absolute Maximization

Time Limit 1000 ms

Mem Limit 262144 kB

You are given an array a of length n . You can perform the following operation several (possibly, zero) times:

- Choose i, j, b : Swap the b -th digit in the binary representation of a_i and a_j .

Find the maximum possible value of $\max(a) - \min(a)$.

In a binary representation, bits are numbered from right (least significant) to left (most significant). Consider that there are an infinite number of leading zero bits at the beginning of any binary representation.

For example, swap the 0-th bit for $4 = 100_2$ and $3 = 11_2$ will result $101_2 = 5$ and $10_2 = 2$. Swap the 2-nd bit for $4 = 100_2$ and $3 = 11_2$ will result $000_2 = 0_2 = 0$ and $111_2 = 7$.

Here, $\max(a)$ denotes the maximum element of array a and $\min(a)$ denotes the minimum element of array a .

The binary representation of x is x written in base 2. For example, 9 and 6 written in base 2 are 1001 and 110, respectively.

Input

The first line contains a single integer t ($1 \leq t \leq 128$) — the number of testcases.

The first line of each test case contains a single integer n ($3 \leq n \leq 512$) — the length of array a .

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i < 1024$) — the elements of array a .

It's guaranteed that the sum of n over all testcases does not exceed 512.

Output

For each testcase, print one integer — the maximum possible value of $\max(a) - \min(a)$.

Examples

Input	Output
4 3 1 0 1 4 5 5 5 5 5 1 2 3 4 5 7 20 85 100 41 76 49 36	1 0 7 125

Note

In the first example, it can be shown that we do not need to perform any operations — the maximum value of $\max(a) - \min(a)$ is $1 - 0 = 1$.

In the second example, no operation can change the array — the maximum value of $\max(a) - \min(a)$ is $5 - 5 = 0$.

In the third example, initially $a = [1, 2, 3, 4, 5]$, we can perform one operation taking $i = 2$, $j = 5$, $b = 1$. The array now becomes $a = [1, 0, 3, 4, 7]$. It can be shown that any further operations do not lead to a better answer — therefore the answer is $\max(a) - \min(a) = 7 - 0 = 7$.

Problem K. XOR Mixup

Time Limit 1000 ms

Mem Limit 262144 kB

There is an array a with $n - 1$ integers. Let x be the [bitwise XOR](#) of all elements of the array. The number x is added to the end of the array a (now it has length n), and then the elements are shuffled.

You are given the newly formed array a . What is x ? If there are multiple possible values of x , you can output any of them.

Input

The input consists of multiple test cases. The first line contains an integer t ($1 \leq t \leq 1000$) — the number of test cases. The description of the test cases follows.

The first line of each test case contains an integer n ($2 \leq n \leq 100$) — the number of integers in the resulting array a .

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 127$) — the elements of the newly formed array a .

Additional constraint on the input: the array a is made by the process described in the statement; that is, some value of x exists.

Output

For each test case, output a single integer — the value of x , as described in the statement. If there are multiple possible values of x , output any of them.

Examples

Input	Output
4	3
4	7
4 3 2 5	6
5	0
6 1 10 7 10	
6	
6 6 6 6 6 6	
3	
100 100 0	

Note

In the first test case, one possible array a is $a = [2, 5, 4]$. Then $x = 2 \oplus 5 \oplus 4 = 3$ ($\oplus$ denotes the bitwise XOR), so the new array is $[2, 5, 4, 3]$. Afterwards, the array is shuffled to form $[4, 3, 2, 5]$.

In the second test case, one possible array a is $a = [1, 10, 6, 10]$. Then $x = 1 \oplus 10 \oplus 6 \oplus 10 = 7$, so the new array is $[1, 10, 6, 10, 7]$. Afterwards, the array is shuffled to form $[6, 1, 10, 7, 10]$.

In the third test case, all elements of the array are equal to 6, so $x = 6$.

In the fourth test case, one possible array a is $a = [100, 100]$. Then $x = 100 \oplus 100 = 0$, so the new array is $[100, 100, 0]$. Afterwards, the array is shuffled to form $[100, 100, 0]$. (Note that after the shuffle, the array can remain the same.)

Problem L. Min Or Sum

Time Limit 1000 ms

Mem Limit 262144 kB

You are given an array a of size n .

You can perform the following operation on the array:

- Choose two different integers i, j ($1 \leq i < j \leq n$), replace a_i with x and a_j with y . In order not to break the array, $a_i | a_j = x | y$ must be held, where $|$ denotes the [bitwise OR operation](#). Notice that x and y are non-negative integers.

Please output the minimum sum of the array you can get after using the operation above any number of times.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 1000$). Description of the test cases follows.

The first line of each test case contains an integer n ($2 \leq n \leq 100$) — the size of array a .

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i < 2^{30}$).

Output

For each test case, print one number in a line — the minimum possible sum of the array.

Examples

Input	Output
4	3
3	31
1 3 2	6
5	7
1 2 4 8 16	
2	
6 6	
3	
3 5 6	

Note

In the first example, you can perform the following operations to obtain the array $[1, 0, 2]$:

1. choose $i = 1, j = 2$, change $a_1 = 1$ and $a_2 = 2$, it's valid since $1|3 = 1|2$. The array becomes $[1, 2, 2]$.

2. choose $i = 2, j = 3$, change $a_2 = 0$ and $a_3 = 2$, it's valid since $2|2 = 0|2$. The array becomes $[1, 0, 2]$.

We can prove that the minimum sum is $1 + 0 + 2 = 3$

In the second example, We don't need any operations.

Problem M. And Then There Were K

Time Limit 1000 ms

Mem Limit 262144 kB

Given an integer n , find the maximum value of integer k such that the following condition holds:

$$n \& (n - 1) \& (n - 2) \& (n - 3) \& \dots \& (k) = 0$$

where $\&$ denotes the [bitwise AND operation](#).

Input

The first line contains a single integer t ($1 \leq t \leq 3 \cdot 10^4$). Then t test cases follow.

The first line of each test case contains a single integer n ($1 \leq n \leq 10^9$).

Output

For each test case, output a single integer — the required integer k .

Examples

Input	Output
3	1
2	3
5	15
17	

Note

In the first testcase, the maximum value for which the continuous $\&$ operation gives 0 value, is 1.

In the second testcase, the maximum value for which the continuous $\&$ operation gives 0 value, is 3. No value greater than 3, say for example 4, will give the $\&$ sum 0.

- $5 \& 4 \neq 0$,
- $5 \& 4 \& 3 = 0$.

Hence, 3 is the answer.

Problem N. Raising Bacteria

Time Limit 1000 ms

Mem Limit 262144 kB

You are a lover of bacteria. You want to raise some bacteria in a box.

Initially, the box is empty. Each morning, you can put any number of bacteria into the box. And each night, every bacterium in the box will split into two bacteria. You hope to see exactly X bacteria in the box at some moment.

What is the minimum number of bacteria you need to put into the box across those days?

Input

The only line containing one integer x ($1 \leq x \leq 10^9$).

Output

The only line containing one integer: the answer.

Examples

Input	Output
5	2

Input	Output
8	1

Note

For the first sample, we can add one bacterium in the box in the first day morning and at the third morning there will be 4 bacteria in the box. Now we put one more resulting 5 in the box. We added 2 bacteria in the process so the answer is 2.

For the second sample, we can put one in the first morning and in the 4-th morning there will be 8 in the box. So the answer is 1.

Problem O. Odd One Out

Time Limit 1000 ms

Mem Limit 262144 kB

You are given three digits a, b, c . Two of them are equal, but the third one is different from the other two.

Find the value that occurs exactly once.

Input

The first line contains a single integer t ($1 \leq t \leq 270$) — the number of test cases.

The only line of each test case contains three digits a, b, c ($0 \leq a, b, c \leq 9$). Two of the digits are equal, but the third one is different from the other two.

Output

For each test case, output the value that occurs exactly once.

Examples

Input	Output
10	1
1 2 2	3
4 3 4	6
5 5 6	7
7 8 8	0
9 0 9	6
3 6 3	8
2 8 2	5
5 7 7	5
7 7 5	7
5 7 5	

Problem P. XOR Sequences

Time Limit 1000 ms

Mem Limit 262144 kB

You are given two distinct non-negative integers x and y . Consider two infinite sequences $a_1, a_2, a_3, \dots$ and $b_1, b_2, b_3, \dots$, where

- $a_n = n \oplus x$;
- $b_n = n \oplus y$.

Here, $x \oplus y$ denotes the [bitwise XOR](#) operation of integers x and y .

For example, with $x = 6$, the first 8 elements of sequence a will look as follows: $[7, 4, 5, 2, 3, 0, 1, 14, \dots]$. Note that the indices of elements start with 1.

Your task is to find the length of the longest common subsegment[†] of sequences a and b . In other words, find the maximum integer m such that $a_i = b_j, a_{i+1} = b_{j+1}, \dots, a_{i+m-1} = b_{j+m-1}$ for some $i, j \geq 1$.

[†]A subsegment of sequence p is a sequence $p_l, p_{l+1}, \dots, p_r$, where $1 \leq l \leq r$.

Input

Each test consists of multiple test cases. The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases. The description of the test cases follows.

The only line of each test case contains two integers x and y ($0 \leq x, y \leq 10^9, x \neq y$) — the parameters of the sequences.

Output

For each test case, output a single integer — the length of the longest common subsegment.

Examples

Input	Output
4 0 1 12 4 57 37 316560849 14570961	1 8 4 33554432

Note

In the first test case, the first 7 elements of sequences a and b are as follows:

$$a = [1, 2, 3, 4, 5, 6, 7, \dots]$$

$$b = [0, 3, 2, 5, 4, 7, 6, \dots]$$

It can be shown that there isn't a positive integer k such that the sequence $[k, k + 1]$ occurs in b as a subsegment. So the answer is 1.

In the third test case, the first 20 elements of sequences a and b are as follows:

$$a = [56, 59, 58, 61, 60, 63, 62, 49, 48, 51, 50, 53, 52, 55, 54, \mathbf{41}, \mathbf{40}, \mathbf{43}, \mathbf{42}, 45, \dots]$$

$$b = [36, 39, 38, 33, 32, 35, 34, 45, 44, 47, 46, \mathbf{41}, \mathbf{40}, \mathbf{43}, \mathbf{42}, 53, 52, 55, 54, 49, \dots]$$

It can be shown that one of the longest common subsegments is the subsegment $[41, 40, 43, 42]$ with a length of 4.

Problem Q. MEXor Mixup

Time Limit 2000 ms

Mem Limit 262144 kB

Alice gave Bob two integers a and b ($a > 0$ and $b \geq 0$). Being a curious boy, Bob wrote down an array of **non-negative** integers with MEX value of all elements equal to a and XOR value of all elements equal to b .

What is the *shortest* possible length of the array Bob wrote?

Recall that the MEX ([Minimum EXcluded](#)) of an array is the minimum non-negative integer that does **not** belong to the array and the XOR of an array is the [bitwise XOR](#) of all the elements of the array.

Input

The input consists of multiple test cases. The first line contains an integer t ($1 \leq t \leq 5 \cdot 10^4$) — the number of test cases. The description of the test cases follows.

The only line of each test case contains two integers a and b ($1 \leq a \leq 3 \cdot 10^5$; $0 \leq b \leq 3 \cdot 10^5$) — the MEX and XOR of the array, respectively.

Output

For each test case, output one (positive) integer — the length of the shortest array with MEX a and XOR b . We can show that such an array always exists.

Examples

Input	Output
5	3
1 1	2
2 1	3
2 0	2
1 10000	3
2 10000	

Note

In the first test case, one of the shortest arrays with MEX 1 and XOR 1 is $[0, 2020, 2021]$.

In the second test case, one of the shortest arrays with MEX 2 and XOR 1 is $[0, 1]$.

It can be shown that these arrays are the shortest arrays possible.

Problem R. MAX-MEX Cut

Time Limit 1000 ms

Mem Limit 262144 kB

A binary string is a string that consists of characters 0 and 1. A bi-table is a table that has exactly two rows of equal length, each being a binary string.

Let MEX of a bi-table be the smallest digit among 0, 1, or 2 that does not occur in the bi-table. For example, MEX for $\begin{bmatrix} 0011 \\ 1010 \end{bmatrix}$ is 2, because 0 and 1 occur in the bi-table at least once.

MEX for $\begin{bmatrix} 111 \\ 111 \end{bmatrix}$ is 0, because 0 and 2 do not occur in the bi-table, and $0 < 2$.

You are given a bi-table with n columns. You should cut it into any number of bi-tables (each consisting of consecutive columns) so that each column is in exactly one bi-table. It is possible to cut the bi-table into a single bi-table — the whole bi-table.

What is the **maximal** sum of MEX of all resulting bi-tables can be?

Input

The input consists of multiple test cases. The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases. Description of the test cases follows.

The first line of the description of each test case contains a single integer n ($1 \leq n \leq 10^5$) — the number of columns in the bi-table.

Each of the next two lines contains a binary string of length n — the rows of the bi-table.

It's guaranteed that the sum of n over all test cases does not exceed 10^5 .

Output

For each test case print a single integer — the maximal sum of MEX of all bi-tables that it is possible to get by cutting the given bi-table optimally.

Examples

Input	Output
4 7 0101000 1101100 5 01100 10101 2 01 01 6 000000 111111	8 8 2 12

Note

In the first test case you can cut the bi-table as follows:

- $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$, its MEX is 2.
- $\begin{bmatrix} 10 \\ 10 \end{bmatrix}$, its MEX is 2.
- $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$, its MEX is 0.
- $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$, its MEX is 2.
- $\begin{bmatrix} 0 \\ 0 \end{bmatrix}$, its MEX is 1.
- $\begin{bmatrix} 0 \\ 0 \end{bmatrix}$, its MEX is 1.

The sum of MEX is 8.

Problem S. Friendly Arrays

Time Limit 2000 ms

Mem Limit 262144 kB

You are given two arrays of integers — $a_1, \dots, a_n$ of length n , and $b_1, \dots, b_m$ of length m . You can choose any element b_j from array b ($1 \leq j \leq m$), and for all $1 \leq i \leq n$ perform $a_i = a_i | b_j$. You can perform any number of such operations.

After all the operations, the value of $x = a_1 \oplus a_2 \oplus \dots \oplus a_n$ will be calculated. Find the minimum and maximum values of x that could be obtained after performing any set of operations.

Above, $|$ is the [bitwise OR operation](#), and $\oplus$ is the [bitwise XOR operation](#).

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases. This is followed by the description of the test cases.

The first line of each test case contains two integers n and m ($1 \leq n, m \leq 2 \cdot 10^5$) — the sizes of arrays a and b .

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^9$) — the array a .

The third line of each test case contains m integers $b_1, b_2, \dots, b_m$ ($0 \leq b_i \leq 10^9$) — the array b .

It is guaranteed that the sum of values of n and m across all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output 2 numbers: the minimum and maximum possible values of x after performing any set of operations.

Examples

Input	Output
2 2 3 0 1 1 2 3 3 1 1 1 2 1	0 1 2 3

Note

In the first test case, if we apply the operation with element $b_1 = 1$, the array a will become $[1, 1]$, and x will be 0. If no operations are applied, then $x = 1$.

In the second test case, if no operations are applied, then $x = 2$. If we apply the operation with $b_1 = 1$, then $a = [1, 1, 3]$, and $x = 3$.

Problem T. Rock and Lever

Time Limit 1000 ms

Mem Limit 262144 kB

"You must lift the dam. With a lever. I will give it to you.

You must block the canal. With a rock. I will not give the rock to you."

Danik urgently needs rock and lever! Obviously, the easiest way to get these things is to ask Hermit Lizard for them.

Hermit Lizard agreed to give Danik the lever. But to get a stone, Danik needs to solve the following task.

You are given a positive integer n , and an array a of positive integers. The task is to calculate the number of such pairs (i, j) that $i < j$ and $a_i \& a_j \geq a_i \oplus a_j$, where $\&$ denotes the [bitwise AND operation](#), and $\oplus$ denotes the [bitwise XOR operation](#).

Danik has solved this task. But can you solve it?

Input

Each test contains multiple test cases.

The first line contains one positive integer t ($1 \leq t \leq 10$) denoting the number of test cases. Description of the test cases follows.

The first line of each test case contains one positive integer n ($1 \leq n \leq 10^5$) — length of the array.

The second line contains n positive integers a_i ($1 \leq a_i \leq 10^9$) — elements of the array.

It is guaranteed that the sum of n over all test cases does not exceed 10^5 .

Output

For every test case print one non-negative integer — the answer to the problem.

Examples

Input	Output
5 5 1 4 3 7 10 3 1 1 1 4 6 2 5 3 2 2 4 1 1	1 3 2 0 0

Note

In the first test case there is only one pair: (4, 7): for it $4 \& 7 = 4$, and $4 \oplus 7 = 3$.

In the second test case all pairs are good.

In the third test case there are two pairs: (6, 5) and (2, 3).

In the fourth test case there are no good pairs.

Problem U. Johnny and His Hobbies

Time Limit 1000 ms

Mem Limit 262144 kB

Among Johnny's numerous hobbies, there are two seemingly harmless ones: applying bitwise operations and sneaking into his dad's office. As it is usually the case with small children, Johnny is unaware that combining these two activities can get him in a lot of trouble.

There is a set S containing very important numbers on his dad's desk. The minute Johnny heard about it, he decided that it's a good idea to choose a **positive** integer k and replace each element s of the set S with $s \oplus k$ ($\oplus$ denotes the [exclusive or](#) operation).

Help him choose such k that Johnny's dad will not see any difference after his son is done playing (i.e. Johnny will get the same set as before playing). It is possible that no such number exists. It is also possible that there are many of them. In such a case, output the smallest one. Note that the order of elements in a set doesn't matter, i.e. set $\{1, 2, 3\}$ equals to set $\{2, 1, 3\}$.

Formally, find the smallest positive integer k such that $\{s \oplus k | s \in S\} = S$ or report that there is no such number.

For example, if $S = \{1, 3, 4\}$ and $k = 2$, new set will be equal to $\{3, 1, 6\}$. If $S = \{0, 1, 2, 3\}$ and $k = 1$, after playing set will stay the same.

Input

In the first line of input, there is a single integer t ($1 \leq t \leq 1024$), the number of test cases. In the next lines, t test cases follow. Each of them consists of two lines.

In the first line there is a single integer n ($1 \leq n \leq 1024$) denoting the number of elements in set S . Second line consists of n **distinct** integers s_i ($0 \leq s_i < 1024$), elements of S .

It is guaranteed that the sum of n over all test cases will not exceed 1024.

Output

Print t lines; i -th line should contain the answer to the i -th test case, the minimal positive

integer k satisfying the conditions or -1 if no such k exists.

Examples

Input	Output
6	1
4	4
1 0 2 3	2
6	-1
10 7 14 8 3 12	-1
2	1023
0 2	
3	
1 2 3	
6	
1 4 6 10 11 12	
2	
0 1023	

Note

In the first test case, the answer is 1 because it is a minimum positive integer and it satisfies all the conditions.

Problem V. Johnny and Another Rating Drop

Time Limit 1000 ms

Mem Limit 262144 kB

The last contest held on Johnny's favorite competitive programming platform has been received rather positively. However, Johnny's rating has dropped again! He thinks that the presented tasks are lovely, but don't show the truth about competitors' skills.

The boy is now looking at the ratings of consecutive participants written in a binary system. He thinks that the more such ratings differ, the more unfair is that such people are next to each other. He defines the difference between two numbers as the number of bit positions, where one number has zero, and another has one (we suppose that numbers are padded with leading zeros to the same length). For example, the difference of $5 = 101_2$ and $14 = 1110_2$ equals to 3, since 0101 and 1110 differ in 3 positions. Johnny defines the unfairness of the contest as the sum of such differences counted for neighboring participants.

Johnny has just sent you the rating sequence and wants you to find the unfairness of the competition. You have noticed that you've got a sequence of **consecutive** integers from 0 to n . That's strange, but the boy stubbornly says that everything is right. So help him and find the desired unfairness for received numbers.

Input

The input consists of multiple test cases. The first line contains one integer t ($1 \leq t \leq 10\,000$) — the number of test cases. The following t lines contain a description of test cases.

The first and only line in each test case contains a single integer n ($1 \leq n \leq 10^{18}$).

Output

Output t lines. For each test case, you should output a single line with one integer — the unfairness of the contest if the rating sequence equals to $0, 1, \dots, n-1, n$.

Examples

Input	Output
5 5 7 11 1 20000000000000	8 11 19 1 39999999999987

Note

For $n = 5$ we calculate unfairness of the following sequence (numbers from 0 to 5 written in binary with extra leading zeroes, so they all have the same length):

- 000
- 001
- 010
- 011
- 100
- 101

The differences are equal to 1, 2, 1, 3, 1 respectively, so unfairness is equal to $1 + 2 + 1 + 3 + 1 = 8$.

Problem W. Skibidi Table

Time Limit 2000 ms

Mem Limit 262144 kB

Vadim loves filling square tables with integers. But today he came up with a way to do it for fun! Let's take, for example, a table of size 2×2 , with rows numbered from top to bottom and columns numbered from left to right. We place 1 in the top left cell, 2 in the bottom right, 3 in the bottom left, and 4 in the top right. That's all he needs for fun!

Fortunately for Vadim, he has a table of size $2^n \times 2^n$. He plans to fill it with integers from 1 to 2^{2n} in ascending order. To fill such a large table, Vadim will divide it into 4 equal square tables, filling the top left one first, then the bottom right one, followed by the bottom left one, and finally the top right one. Each smaller table will be divided into even smaller ones as he fills them until he reaches tables of size 2×2 , which he will fill in the order described above.

Now Vadim is eager to start filling the table, but he has q questions of two types:

- what number will be in the cell at the x -th row and y -th column;
- in which cell coordinates will the number d be located.

Help answer Vadim's questions.

Input

Each test consists of several sets of input data. The first line contains a single integer t ($1 \leq t \leq 10$) — the number of sets of input data. The following lines describe the input data sets.

In the first line of each data set, there is an integer n , describing the size of the table ($1 \leq n \leq 30$).

In the second line of each data set, there is an integer q — the number of questions ($1 \leq q \leq 20\,000$).

In the following q lines of each data set, the questions are described in the following formats:

- $\rightarrow x\ y$ — What number will be in the cell ($1 \leq x, y \leq 2^n$);
- $< d$ — In which cell coordinates will the number ($1 \leq d \leq 2^{2n}$) be located.

It is guaranteed that the sum of q over all test cases does not exceed 20 000.

Output

Output the answers to each question on a separate line.

Examples

Input	Output
2	7
2	2 3
5	1 2
-> 4 3	9
<- 15	13
<- 4	1
-> 3 1	4
-> 1 3	3
1	2
8	1 1
-> 1 1	2 2
-> 1 2	2 1
-> 2 1	1 2
-> 2 2	
<- 1	
<- 2	
<- 3	
<- 4	

Note

This is how the filled table from the first example looks:

1	4	13	16
3	2	15	14
9	12	5	8
11	10	7	6

Problem X. Fortune Telling

Time Limit 1000 ms

Mem Limit 262144 kB

Haha, try to solve this, SelectorUnlimited!

— antontrygubO_o

Your friends Alice and Bob practice fortune telling.

Fortune telling is performed as follows. There is a well-known array a of n non-negative integers indexed from 1 to n . The tellee starts with some non-negative number d and performs one of the two operations for each $i = 1, 2, \dots, n$, **in the increasing order of i** . The possible operations are:

- replace their current number d with $d + a_i$
- replace their current number d with $d \oplus a_i$ (hereinafter $\oplus$ denotes the [bitwise XOR operation](#))

Notice that the chosen operation may be different for different i and for different telleeps.

One time, Alice decided to start with $d = x$ and Bob started with $d = x + 3$. Each of them performed fortune telling and got a particular number in the end. Notice that the friends chose operations independently of each other, that is, they could apply different operations for the same i .

You learnt that either Alice or Bob ended up with number y in the end, but you don't know whose of the two it was. Given the numbers Alice and Bob started with and y , find out who (Alice or Bob) could get the number y after performing the operations. It is guaranteed that on the jury tests, **exactly one** of your friends could have actually gotten that number.

Hacks

You cannot make hacks in this problem.

Input

On the first line of the input, you are given one number t ($1 \leq t \leq 10^4$) — the number of test cases. The following $2 \cdot t$ lines contain test cases.

The first line of each test case contains three numbers n, x, y ($1 \leq n \leq 10^5, 0 \leq x \leq 10^9, 0 \leq y \leq 10^{15}$) — the length of array a , Alice's initial number (Bob's initial number is

therefore $x + 3$), and the number that one of the two friends got in the end.

The second line of each test case contains n numbers — the array a ($0 \leq a_i \leq 10^9$).

It is guaranteed that the sum of n over all test cases does not exceed 10^5 .

Output

For each test case, print the name of the friend who could get the number y : "Alice" or "Bob".

Examples

Input	Output
4 1 7 9 2 2 0 2 1 3 4 0 1 1 2 3 4 2 1000000000 3000000000 1000000000 1000000000	Alice Alice Bob Alice

Note

In the first test case, Alice could get 9 using the following operations: $7 + 2 = 9$.

In the second test case, Alice could get 2 using this operations: $(0 + 1) \oplus 3 = 2$.

In the third test case, Bob started with $x + 3 = 0 + 3 = 3$ and could get 1 this way: $((3 + 1) + 2) \oplus 3) \oplus 4 = 1$.

Problem Y. Flexible String

Time Limit 2000 ms

Mem Limit 262144 kB

You have a string a and a string b . Both of the strings have length n . There are **at most 10 different characters** in the string a . You also have a set Q . Initially, the set Q is empty. You can apply the following operation on the string a any number of times:

- Choose an index i ($1 \leq i \leq n$) and a lowercase English letter c . Add a_i to the set Q and then replace a_i with c .

For example, Let the string a be "abecca". We can do the following operations:

- In the first operation, if you choose $i = 3$ and $c = \mathbf{x}$, the character $a_3 = \mathbf{e}$ will be added to the set Q . So, the set Q will be $\{\mathbf{e}\}$, and the string a will be "abxcca".
- In the second operation, if you choose $i = 6$ and $c = \mathbf{s}$, the character $a_6 = \mathbf{a}$ will be added to the set Q . So, the set Q will be $\{\mathbf{e}, \mathbf{a}\}$, and the string a will be "abxcccs".

You can apply any number of operations on a , but in the end, the set Q should contain **at most k different characters**. Under this constraint, you have to maximize the number of integer pairs (l, r) ($1 \leq l \leq r \leq n$) such that $a[l, r] = b[l, r]$. Here, $s[l, r]$ means the substring of string s starting at index l (inclusively) and ending at index r (inclusively).

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 10^4$). The description of the test cases follows.

The first line contains two integers n and k ($1 \leq n \leq 10^5$, $0 \leq k \leq 10$) — the length of the two strings and the limit on different characters in the set Q .

The second line contains the string a of length n . There is **at most 10 different characters** in the string a .

The last line contains the string b of length n .

Both of the strings a and b contain only lowercase English letters. The sum of n over all test cases doesn't exceed 10^5 .

Output

For each test case, print a single integer in a line, the maximum number of pairs (l, r) satisfying the constraints.

Examples

Input	Output
6 3 1 abc abd 3 0 abc abd 3 1 xbb xcd 4 1 abcd axcb 3 10 abc abd 10 3 lkwhbahuqa qoiujoncjb	6 3 6 6 6 11

Note

In the first case, we can select index $i = 3$ and replace it with character $c = d$. All possible pairs (l, r) will be valid.

In the second case, we can't perform any operation. The 3 valid pairs (l, r) are:

1. $a[1, 1] = b[1, 1] = \text{"a"}$,
2. $a[1, 2] = b[1, 2] = \text{"ab"}$,
3. $a[2, 2] = b[2, 2] = \text{"b"}$.

In the third case, we can choose index 2 and index 3 and replace them with the characters c and d respectively. The final set Q will be $\{b\}$ having size 1 that satisfies the value of k . All possible pairs (l, r) will be valid.

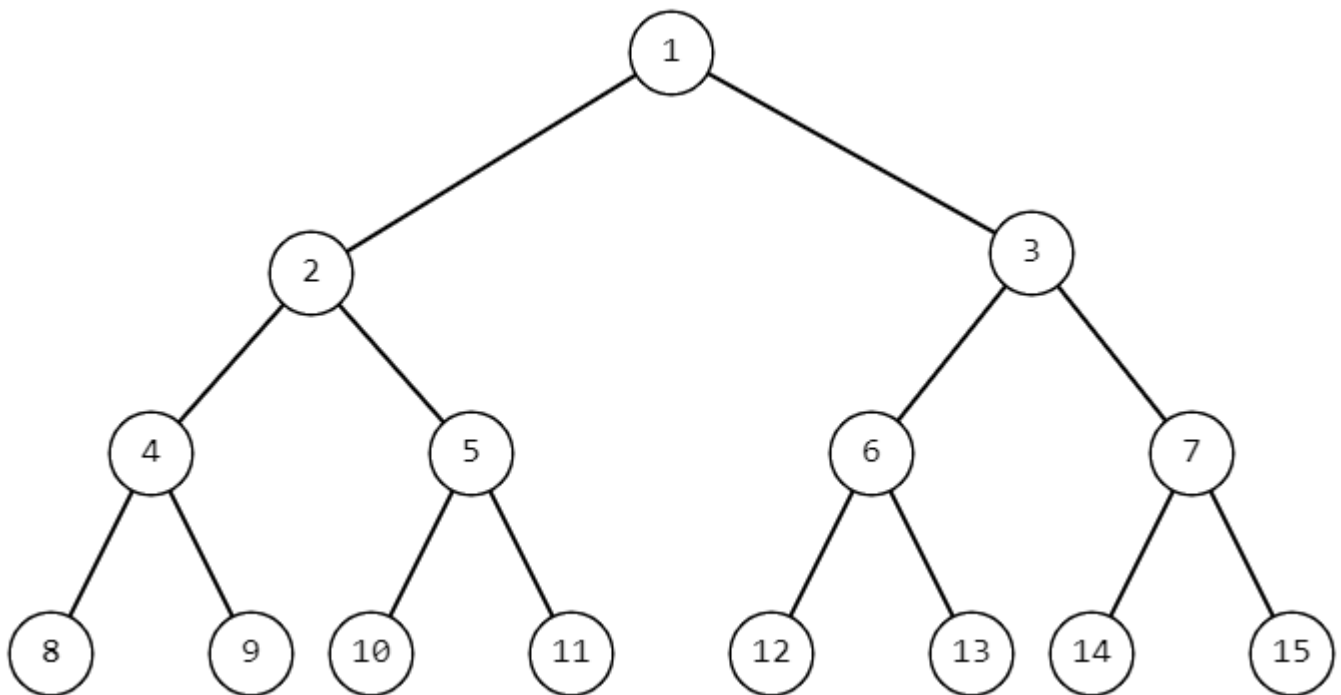
Problem Z. Sum in Binary Tree

Time Limit 1000 ms

Mem Limit 262144 kB

Vanya really likes math. One day when he was solving another math problem, he came up with an interesting tree. This tree is built as follows.

Initially, the tree has only one vertex with the number 1 — the root of the tree. Then, Vanya adds two children to it, assigning them consecutive numbers — 2 and 3, respectively. After that, he will add children to the vertices in increasing order of their numbers, starting from 2, assigning their children the minimum unused indices. As a result, Vanya will have an infinite tree with the root in the vertex 1, where each vertex will have exactly two children, and the vertex numbers will be arranged sequentially by layers.



Part of Vanya's tree.

Vanya wondered what the sum of the vertex numbers on the path from the vertex with number 1 to the vertex with number n in such a tree is equal to. Since Vanya doesn't like counting, he asked you to help him find this sum.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

This is followed by t lines — the description of the test cases. Each line contains one integer n ($1 \leq n \leq 10^{16}$) — the number of vertex for which Vanya wants to count the sum of vertex numbers on the path from the root to that vertex.

Output

For each test case, print one integer — the desired sum.

Examples

Input	Output
6 3 10 37 1 100000000000000000 15	4 18 71 1 199999999999999980 26

Note

In the first test case of example on the path from the root to the vertex 3 there are two vertices 1 and 3, their sum equals 4.

In the second test case of example on the path from the root to the vertex with number 10 there are vertices 1, 2, 5, 10, sum of their numbers equals $1 + 2 + 5 + 10 = 18$.